

ROCm-Mobile

A Full-Stack Open Compute Framework for Unsupported AMD Mobile GPUs

Parth Aditya · University of Wisconsin-Madison · CS + Math (B.S., Dec 2026)
paditya2@wisc.edu

Project Overview imes- Following Up on Our Conversation

May 2025 *imes*- November 2025

Overview

This document is the complete written reference for the project we discussed during office hours. It covers every technical layer, all source repositories, and all relevant documentation in one place.

The central problem: AMD's ROCm platform *imes*- the open-source counterpart to NVIDIA's CUDA *imes*- officially supports only a narrow set of desktop GPUs. AMD mobile GPUs across RDNA2 and RDNA3 generations are entirely excluded. The RX 6800M in my ASUS ROG Strix G15 (gfx1031, RDNA2 mobile) is a representative case: the only existing workaround was setting `HSA_OVERRIDE_GFX_VERSION=10.3.0` to lie to the runtime about the hardware identity. As of ROCm 6.4.3, even that workaround is broken *imes*- it now causes `SIGSEGV` crashes, leaving gfx1031 users with no functional path to GPU compute whatsoever.

The affected hardware is not niche. RDNA2 and RDNA3 mobile GPUs appear in millions of laptops sold between 2021 and 2024. Users running JAX, PyTorch, or any ROCm-dependent framework on this hardware are restricted to CPU execution or cloud compute, with no community-maintained solution in place.

This project addresses the problem across the complete software stack *imes*- from the Linux kernel driver through math libraries, ML framework backends, and GPU kernel primitives *imes*- with real upstream contributions at every level. It is structured around six months of focused development and targets a technical report suitable for workshop submission.

The work is intentionally OS-heavy. Kernel module development, character device drivers, memory management policy, GPU scheduler design, and hardware interrupt handling all appear as first-class contributions rather than supporting infrastructure.

Target Hardware Coverage

The RX 6800M is the development and testing target, but the framework is designed to be architecture-agnostic from the outset. The same gap exists across all AMD mobile generations:

Architecture	Desktop	Mobile	Project Coverage
RDNA2	gfx1030 <i>eckmark</i>	gfx1031 <i>imes</i>	Primary target
RDNA3	gfx1100 <i>eckmark</i>	gfx1101 <i>imes</i>	Extension target
RDNA4	gfx1200 <i>eckmark</i>	Mobile pending	Future extension

The GPU capability inference engine (Layer 3) is designed to detect and configure any unsupported AMD GPU based on actual hardware characteristics rather than a hardcoded device list, making the framework extensible to future architectures without code changes.

The Full Software Stack

The following diagram shows every layer this project touches, from bare hardware to Python ML code:

JAX / Optax	PyTorch / Triton	(Python ML frameworks)
Flash Attention kernels (Pallas / HIP, gfx1031-specific)		
XLA compiler backend (mobile capability profile)		
ROCm HIP / ROCr HSA Runtime (capability shim layer)		
rocBLAS / MIOpen (tuned math libraries for mobile)		
/dev/rocm-mobile (character device driver, ioctl interface)		
GPU Memory Tiering Manager (VRAM ↔ system RAM, kernel-level)		
amdgpu: PM subsystem, DRM scheduler, TTM allocator (Linux kernel)		
RX 6800M / RDNA2_{imes} RDNA3 Mobile Hardware		

The project is structured across ten layers. Layers 2 through 5 are entirely kernel-level contributions.

Layer 1 *imes*- Hardware Characterization and Benchmarking

Before optimizing anything, ground truth measurements are required. The `rocm-mobile-bench` suite measures:

- Peak memory bandwidth (GB/s) via custom HIP kernels
- Sustained FLOPS at FP32, FP16, and BF16 under thermal load
- Clock frequency decay over time under sustained ML workloads
- PCIe bandwidth between discrete VRAM and system RAM (critical for Layer 5)
- JAX and PyTorch workload benchmarks: matrix multiply, attention, forward+backward passes

These measurements establish the performance gap between mobile gfx1031 and the officially supported desktop targets, and provide the before/after comparison that anchors every subsequent

layer's contribution. All results are published on GitHub.

Key documentation:

- ROCprofiler-SDK: <https://rocm.docs.amd.com/projects/rocprofiler-sdk/en/latest/>
- HIP programming guide: <https://rocm.docs.amd.com/projects/HIP/en/latest/>
- AMD RDNA2 ISA reference: <https://gpuopen.com/documentation/rdna2-isa-documentation/>

Layer 2 *imes*- Kernel: Power Management, DRM Scheduler, and Sysfs Telemetry

This layer covers three interconnected kernel contributions that collectively give the `amdgpu` driver genuine awareness of mobile ML compute workloads.

2a. Mobile Power Management Patch. The `amdgpu` driver manages GPU power states through the SMU (System Management Unit) interface. The current driver applies the same power state machine to mobile RDNA2 as to desktop RDNA2, despite the 6800M having a 145W TGP. Under a sustained ML training job, the GPU boosts to its maximum clock, hits the thermal ceiling after roughly sixty seconds, and throttles hard. This patch adds a `compute_mobile` power profile in `drivers/gpu/drm/amd/pm/swsmu/smu13/` that targets a conservative sustained clock frequency appropriate for the mobile thermal envelope, preventing the boost-then-throttle behavior.

2b. Workload-Aware DRM GPU Scheduler Extension. The DRM GPU scheduler (`drivers/gpu/drm/scheduler/`) dispatches compute jobs with no concept of workload type. A JAX gradient pass and a gaming frame are treated identically. This extension adds a workload classification layer that distinguishes sustained compute (long-running, regular dispatch cadence, no deadline) from interactive workloads (bursty, latency-sensitive). When a sustained compute context is detected, the scheduler automatically applies the `compute_mobile` power profile. A new `ioctl` interface also allows the ROCm runtime to declare workload intent explicitly as a hint to the scheduler.

2c. Sysfs Telemetry Subsystem. A telemetry interface is exposed under `/sys/class/drm/card0/device/rocm` providing real-time kernel-sourced data: `workload_class`, `sustained_clock_mhz`, `thermal_headroom_c`, `memory_bandwidth_util`, `vram_tiering_pressure`. This makes benchmark measurements significantly more accurate and gives the Layer 5 tiering manager the kernel-level signals it needs to make prefetch decisions.

Key documentation:

- Linux `amdgpu` driver: <https://github.com/torvalds/linux/tree/master/drivers/gpu/drm/amd>
- DRM GPU scheduler source: <https://github.com/torvalds/linux/tree/master/drivers/gpu/drm/scheduler>
- AMD staging kernel branch: <https://gitlab.freedesktop.org/agd5f/linux>
- `amdgpu` kernel docs: <https://docs.kernel.org/gpu/amdgpu/index.html>

- DRM subsystem internals: <https://docs.kernel.org/gpu/drm-internals.html>
- Linux sysfs: <https://docs.kernel.org/filesystems/sysfs.html>
- amd-gfx mailing list: <https://lists.freedesktop.org/archives/amd-gfx/>

Layer 3 *imes*- Kernel: /dev/rocm-mobile Character Device Driver

Rather than intercepting HSA runtime calls via an LD_PRELOAD shim, this layer implements a proper Linux character device driver. The driver exposes /dev/rocm-mobile and provides an `ioctl` interface through which the ROCm userspace runtime queries mobile GPU capabilities, declares workload intent, and registers for thermal events.

The driver integrates with three existing kernel subsystems: the `amdgpu` DRM driver (for hardware capability data read directly from kernel structures), the `hwmon` subsystem (for interrupt-driven thermal events rather than userspace polling), and the Layer 2 DRM scheduler (to forward workload classification hints downward). The LD_PRELOAD shim is retained as a documented fallback.

The driver also implements the GPU capability inference engine: rather than hardcoding support for gfx1031, it reads actual hardware parameters via HSA topology interfaces and derives the correct capability profile automatically. This is what makes the framework extensible to RDNA3 mobile and future architectures without code changes per device.

JAX / ROCm userspace
<code>ioctl(fd, ROCM_MOBILE_GET_CAPS, &caps) ← thin userspace shim</code>
/dev/rocm-mobile (character device, kernel space)
<code>amdgpu + hwmon + DRM scheduler</code>

Key documentation:

- Linux driver API: <https://docs.kernel.org/driver-api/index.html>
- `hwmon` kernel API: <https://docs.kernel.org/hwmon/hwmon-kernel-api.html>
- ROCr HSA Runtime source: <https://github.com/ROCm/ROCR-Runtime>
- HSA Runtime specification: <https://github.com/HSAFoundation/HSA-Runtime-Reference-Source>
- udev rules reference: <https://man7.org/linux/man-pages/man7/udev.7.html>

Layer 4 *imes*- Kernel: Mobile-Aware TTM GPU Memory Allocator

The `amdgpu` driver manages GPU memory through TTM (Translation Table Manager) with AMD's buffer object layer (`amdgpu_bo`) on top. TTM handles allocation of GPU VRAM, eviction when VRAM is under pressure, and migration between memory domains.

The current TTM eviction policy treats all buffer objects equivalently. ML workloads have predictable access patterns: large tensor allocations are long-lived and reused frequently, while intermediate buffers are transient. This layer modifies the eviction priority function to prefer evicting transient buffers before persistent tensor allocations, reducing costly VRAM reload cycles

during training. Large tensor buffers are tagged with a `AMDGPU_BO_ML_TENSOR` flag at allocation time via the `ioctl` interface of Layer 3.

Key documentation:

- TTM source: <https://github.com/torvalds/linux/tree/master/drivers/gpu/drm/ttm>
- `amdgpu_bo` source: <https://github.com/torvalds/linux/tree/master/drivers/gpu/drm/amd/amdgpu>
- DRM memory management: <https://docs.kernel.org/gpu/drm-mm.html>

Layer 5 *imes*- Kernel: GPU Memory Tiering Manager

This is the most OS-research-oriented contribution in the project. On laptops with a discrete GPU, the system has two memory pools available for ML workloads: discrete VRAM (fast, limited *imes*- 12GB GDDR6 at ~192 GB/s on the 6800M) and system RAM accessible via PCIe (larger, slower *imes*- up to 64GB at ~32 GB/s). No existing ROCm component manages the relationship between these two pools intelligently for ML workloads.

The GPU Memory Tiering Manager is a kernel-level subsystem that treats VRAM and system RAM as a two-tier memory hierarchy, directly analogous to classical OS virtual memory with a page replacement policy *imes*- except the “pages” are neural network layers and the “disk” is system RAM. Concretely:

- Model weights are partitioned into **layers** tagged as hot (currently computing), warm (recently used), or cold (not needed in the near future)
- The tiering manager **prefetches** the next hot layer into VRAM via DMA while the current layer is executing, overlapping compute and memory transfer and hiding the PCIe latency entirely
- When VRAM pressure is high, cold layers are **evicted** to pinned system RAM using the Layer 4 buffer tagging to avoid evicting the wrong objects
- The Layer 2 sysfs telemetry interface exposes `vram_tiering_pressure` so userspace tools can observe and tune the policy
- A new `ioctl` command (`ROCM_MOBILE_TIER_HINT`) allows the ML runtime to communicate the model’s layer execution order as a prefetch hint

Practical impact: This enables running models substantially larger than VRAM capacity on AMD mobile hardware. A 70B parameter model at INT4 quantization requires approximately 35GB *imes*- well beyond 12GB VRAM alone, but feasible with intelligent tiering across VRAM and 32GB system RAM. This capability does not currently exist in any AMD-supported tool on this class of hardware.

The prefetch algorithm uses a variant of the classical LRU replacement policy adapted for the sequential access pattern of transformer inference. The eviction and prefetch decisions run in a kernel workqueue (process context, can sleep and allocate), while thermal and pressure signals from the Layer 2 sysfs interface feed into the policy dynamically.

Key documentation:

- TTM memory domains and migration: <https://docs.kernel.org/gpu/drm-mm.html>
- ROCm unified memory concepts: https://rocm.docs.amd.com/projects/HIP/en/latest/how-to/unified_memory.html
- Linux DMA engine API: <https://docs.kernel.org/driver-api/dmaengine/index.html>
- Linux workqueue documentation: <https://docs.kernel.org/core-api/workqueue.html>

Layer 6 *imes*- Math Library Tuning: rocBLAS and MIOpen

JAX and PyTorch dispatch matrix multiplication and convolution operations through AMD's math libraries: rocBLAS and MIOpen. These libraries contain precomputed tuning databases *imes*- tables mapping GPU target and matrix shape to optimal kernel configuration. The RX 6800M mobile is absent from these databases, causing every matrix multiply and convolution to use a generic fallback kernel.

This layer runs AMD's Tensile autotuning framework targeting `gfx1031` with mobile-accurate memory bandwidth constraints, generates a `gfx1031-mobile` tuning database for the matrix shapes JAX and PyTorch use in practice, and runs the MIOpen tuning driver for standard convolution configurations. Both tuning databases are submitted as pull requests to upstream ROCm. Based on comparable work for other non-officially-supported GPUs, gains of 15*imes*35% on compute-bound operations are realistic from tuning alone.

Key documentation:

- rocBLAS source: <https://github.com/ROCm/rocBLAS>
- Tensile autotuning: <https://github.com/ROCm/Tensile>
- MIOpen source: <https://github.com/ROCm/MIOpen>
- MIOpen docs: <https://rocm.docs.amd.com/projects/MIOpen/en/latest/>

Layer 7 *imes*- XLA: Mobile-Aware Compiler Configuration

JAX uses XLA (Accelerated Linear Algebra) to compile Python operations into GPU machine code. XLA queries the device for hardware capabilities *imes*- compute units, wavefront size, shared memory capacity, memory bandwidth *imes*- and uses these to make compilation decisions: tile sizes, vectorization width, memory prefetch depth, and operator fusion strategy.

Because ROCm does not properly report the RX 6800M's capabilities, XLA receives incorrect device parameters and generates suboptimal code for every operation. Layer 3's kernel driver corrects the capability report at the hardware interface level. Layer 7 completes the fix at the compiler level by patching the JAX ROCm backend to accept a mobile GPU capability profile, and adding a `jax.config` flag (`jax_rocm_mobile_mode`) that instructs XLA to prefer memory-bandwidth-efficient operator configurations appropriate for the mobile memory bus.

Key documentation:

- ROCm JAX repository (current, renamed): <https://github.com/ROCm/rocm-jax>
- XLA source (OpenXLA): <https://github.com/openxla/xla>
- XLA documentation: <https://openxla.org/xla>
- JAX build from source: <https://jax.readthedocs.io/en/latest/developer.html>
- ROCm JAX installation: <https://rocm.docs.amd.com/projects/install-on-linux/en/latest/install/3rd-party/jax-install.html>

Layer 8 *imes*- Pallas and Flash Attention: Hand-Optimized Kernels for gfx1031

This layer addresses two interconnected gaps: the absence of ROCm-compatible Flash Attention for consumer RDNA2, and the immaturity of JAX Pallas support on AMD hardware.

8a. Flash Attention for gfx1031. Flash Attention is the algorithm underpinning every modern transformer model *imes*- it enables memory-efficient attention by fusing the softmax and matrix multiply operations into a single kernel, avoiding materializing the full attention matrix in VRAM. NVIDIA has a highly optimized Flash Attention implementation. AMD has `composable_kernel` which includes a Flash Attention port, but it is poorly optimized for consumer RDNA2 and produces incorrect results on gfx1031. Without a working Flash Attention, efficient inference of any modern LLM or vision transformer on AMD mobile hardware is not possible.

This contribution implements Flash Attention targeting gfx1031 using JAX Pallas, benchmarks it against the `composable_kernel` fallback, and characterizes the performance gap. A working, correct, benchmarked Flash Attention implementation on RDNA2 mobile is a novel contribution with no published precedent.

8b. Pallas_{imes}ROCm Interface Characterization. The JAX issue tracker explicitly labels Pallas ROCm support as “contributions welcome” *imes*- meaning the JAX team has not prioritized this work. Beyond the Flash Attention kernel, this contribution systematically documents where the Pallas_{imes}ROCm interface breaks for gfx1031, what fixes are required, and submits those fixes as pull requests to ROCm/rocm-jax. This positions the project as a meaningful upstream contributor to the AMD JAX ecosystem.

Key documentation:

- Pallas documentation: <https://jax.readthedocs.io/en/latest/pallas/index.html>
- ROCm JAX repository: <https://github.com/ROCm/rocm-jax>
- `composable_kernel` (AMD Flash Attention reference): https://github.com/ROCm/composable_kernel
- Flash Attention original paper (Dao et al., 2022): <https://arxiv.org/abs/2205.14135>
- Flash Attention 2 (Dao, 2023): <https://arxiv.org/abs/2307.08691>

Layer 9 *imes*- Triton: PyTorch Kernel Compilation for gfx1031

OpenAI's Triton is the kernel compilation backend for PyTorch 2.0's `torch.compile` *imes*- the mechanism that automatically fuses and optimizes PyTorch operations for the target hardware. AMD has been developing Triton ROCm support, but consumer mobile GPUs are essentially untested targets. Getting Triton to generate correct, optimized code for gfx1031 unlocks the entire PyTorch 2.x ecosystem *imes*- automatic kernel fusion, operator optimization, and `torch.compile` *imes*- on AMD mobile hardware.

This layer is the PyTorch-ecosystem equivalent of Layer 7's JAX work. The two layers together mean the project covers the full open-source ML landscape: JAX users (Layer 7/8) and PyTorch users (Layer 9) both benefit. The user base for PyTorch is substantially larger than JAX, making this the highest-reach contribution in the stack.

Concretely, this involves: identifying where Triton's gfx1031 code generation fails, fixing the relevant LLVM AMDGPU backend configurations, validating `torch.compile` end-to-end on standard model architectures, and submitting fixes upstream to the Triton ROCm repository.

Key documentation:

- Triton source: <https://github.com/triton-lang/triton>
- ROCm Triton fork: <https://github.com/ROCm/triton>
- PyTorch ROCm source: <https://github.com/pytorch/pytorch>
- Optax (JAX optimizer library): <https://github.com/google-deepmind/optax>

Layer 10 *imes*- End-to-End Validation and Community Release

The final layer integrates and validates all preceding layers as a cohesive, installable stack:

- A single-command install: `pip install rocm-mobile` or a shell installer
- End-to-end validation: llama.cpp inference, Stable Diffusion, a JAX transformer training loop, and a PyTorch `torch.compile` workload *imes*- all running correctly on gfx1031
- A community-contributed GPU capability database supporting RDNA3 mobile variants (gfx1101) in addition to the RDNA2 primary target
- A technical report (8*imes*12 pages) structured for workshop submission, covering problem statement, system design, per-layer benchmarks, and results

The demo target: a 70B parameter quantized model running locally on an AMD laptop GPU that previously had no GPU ML support at all.

Feasibility Assessment and Estimated Scope

Layer	Component	Notes
1	Benchmarking	Fully feasible; HIP and Python, no unknowns
2	PM, DRM scheduler, sysfs	PM patch and sysfs well-scoped; scheduler extension is the highest-risk single component
3	Character device driver	Moderate; standard pattern, well-documented; harder than LD_PRELOAD but architecturally correct
4	TTM eviction policy	Moderate-high; scoped to eviction policy modification, not a full allocator rewrite
5	GPU memory tiering	High; most novel OS contribution; prefetch correctness under concurrent access requires careful locking
6	rocBLAS / MIOpen tuning	Feasible; mostly automated GPU compute time; start early
7	XLA / JAX config	Moderate; large codebase, localized changes
8	Pallas / Flash Attention	High; Flash Attention correctness on RDNA2 is non-trivial; scoped to working implementation + benchmarks
9	Triton / torch.compile	Moderate-high; extends to PyTorch ecosystem; LLVM AMDGPU backend knowledge required
10	Integration and release	Moderate; primarily packaging, documentation, and validation

Estimated lines of code:

Component	Estimated LOC
Layer 1: rocm-mobile-bench (HIP + Python)	~2,000
Layer 2: PM patch, DRM scheduler, sysfs	~1,800
Layer 3: /dev/rocm-mobile driver + userspace shim	~3,000
Layer 4: TTM eviction policy	~900
Layer 5: GPU memory tiering manager	~2,500
Layer 6: Math library tuning pipeline	~800
Layer 7: XLA / JAX mobile config	~500
Layer 8: Pallas kernels + Flash Attention	~1,800
Layer 9: Triton gfx1031 fixes	~1,200
Layer 10: Integration, packaging, docs	~2,000
Total	~16,500

Approximately 8,200 lines reside in kernel space (Layers 2*imes*5), reflecting the OS-heavy orientation of the project.

Planned Deliverables

Deliverable	Description
<code>rocm-mobile-bench</code>	Open-source benchmarking suite; full baseline characterization published on GitHub
Layer 2 kernel patch set	PM profile, DRM scheduler extension, sysfs telemetry; submitted to <code>amd-gfx</code> mailing list
<code>/dev/rocm-mobile driver</code>	Character device driver with <code>ioctl</code> interface and capability inference engine
TTM eviction policy patch	Mobile ML-aware buffer eviction; submitted upstream
GPU memory tiering manager	Kernel subsystem enabling VRAM↔RAM tiering with prefetch; submitted upstream
<code>gfx1031-mobile tuning DB</code>	rocBLAS and MIOpen tuning database; pull request to upstream ROCm
JAX mobile config patch	Pull request to ROCm/ <code>rocm-jax</code> ; corrected XLA capability profile
Flash Attention / Pallas kernels	Working Flash Attention for <code>gfx1031</code> ; Pallas <code>imes</code> ROCm interface fixes submitted upstream
Triton <code>gfx1031</code> fixes	Working <code>torch.compile</code> on AMD mobile; pull requests to ROCm/ <code>triton</code>
<code>rocm-mobile installable package</code>	Single-command install; GPU capability database supporting RDNA2 and RDNA3 mobile
Technical report	<i>8imes</i> 12 page writeup structured for workshop submission

Projected Timeline

May <i>imes</i> June	Development environment setup (Ubuntu 22.04 dual boot, ROCm, JAX and PyTorch baselines). Layer 1 benchmarking suite; full baseline gap analysis. Tensile autotuning initiated in parallel (GPU-compute-intensive).
July	Core kernel development: Layer 2 patch set (PM, DRM scheduler, sysfs) and Layer 3 character device driver. These are the heaviest and highest-risk components.
August	Layer 4 TTM eviction policy. Layer 5 GPU memory tiering manager <i>imes</i> -the most research-novel kernel contribution. Layer 6 math library tuning finalized.
September	Layer 7 XLA/JAX configuration patch. Layer 8 Flash Attention implementation in Pallas. End-to-end JAX validation.
October	Layer 9 Triton <code>gfx1031</code> fixes and <code>torch.compile</code> validation. Layer 10 integration and packaging. Upstream PR submissions across all layers.
November	Technical report. Demo: 70B parameter model inference on AMD mobile

hardware. Community release.

Quick Reference: All Documentation and Source Links

Kernel and Driver

- Linux amdgpu driver: <https://github.com/torvalds/linux/tree/master/drivers/gpu/drm/amd>
- DRM GPU scheduler: <https://github.com/torvalds/linux/tree/master/drivers/gpu/drm/scheduler>
- TTM memory manager: <https://github.com/torvalds/linux/tree/master/drivers/gpu/drm/ttm>
- AMD staging kernel branch: <https://gitlab.freedesktop.org/agd5f/linux>
- amdgpu kernel docs: <https://docs.kernel.org/gpu/amdgpu/index.html>
- DRM subsystem internals: <https://docs.kernel.org/gpu/drm-internals.html>
- DRM memory management (TTM): <https://docs.kernel.org/gpu/drm-mm.html>
- Linux driver API (char devices): <https://docs.kernel.org/driver-api/index.html>
- hwmon kernel API: <https://docs.kernel.org/hwmon/hwmon-kernel-api.html>
- Linux DMA engine API: <https://docs.kernel.org/driver-api/dmaengine/index.html>
- Linux workqueue documentation: <https://docs.kernel.org/core-api/workqueue.html>
- Linux sysfs: <https://docs.kernel.org/filesystems/sysfs.html>
- amd-gfx mailing list: <https://lists.freedesktop.org/archives/amd-gfx/>
- AMD RDNA2 ISA reference: <https://gpuopen.com/documentation/rdna2-isa-documentation/>
- Linux kernel cross-reference (Elixir): <https://elixir.bootlin.com/linux/latest/source>

ROCm Runtime

- ROCm documentation hub: <https://rocm.docs.amd.com/en/latest/>
- ROCr HSA Runtime source: <https://github.com/ROCm/ROCR-Runtime>
- HIP Runtime source: <https://github.com/ROCm/HIP>
- HIP programming guide: <https://rocm.docs.amd.com/projects/HIP/en/latest/>
- HSA Runtime specification (GitHub mirror): <https://github.com/HSAFoundation/HSA-Runtime-Reference>
- ROCm Linux installation guide: <https://rocm.docs.amd.com/projects/install-on-linux/en/latest/>

- ROCm unified memory: https://rocm.docs.amd.com/projects/HIP/en/latest/how-to/unified_memory.html
- udev rules reference: <https://man7.org/linux/man-pages/man7/udev.7.html>

Math Libraries

- rocBLAS source: <https://github.com/ROCm/rocBLAS>
- Tensile autotuning: <https://github.com/ROCm/Tensile>
- MIOpen source: <https://github.com/ROCm/MIOpen>
- composable_kernel (Flash Attention reference): https://github.com/ROCm/composable_kernel
- MIOpen docs: <https://rocm.docs.amd.com/projects/MIOpen/en/latest/>
- rocBLAS docs: <https://rocm.docs.amd.com/projects/rocBLAS/en/latest/>

JAX / XLA / Pallas / Flash Attention

- ROCm JAX repository: <https://github.com/ROCm/rocm-jax>
- XLA source (OpenXLA): <https://github.com/openxla/xla>
- XLA documentation: <https://openxla.org/xla>
- Pallas documentation: <https://jax.readthedocs.io/en/latest/pallas/index.html>
- JAX developer guide: <https://jax.readthedocs.io/en/latest/developer.html>
- JAX ROCm install guide: <https://rocm.docs.amd.com/projects/install-on-linux/en/latest/install/3rd-party/jax-install.html>
- Flash Attention paper (Dao et al., 2022): <https://arxiv.org/abs/2205.14135>
- Flash Attention 2 (Dao, 2023): <https://arxiv.org/abs/2307.08691>
- Optax: <https://github.com/google-deepmind/optax>

PyTorch / Triton

- Triton source: <https://github.com/triton-lang/triton>
- ROCm Triton fork: <https://github.com/ROCm/triton>
- PyTorch source: <https://github.com/pytorch/pytorch>
- PyTorch ROCm documentation: <https://rocm.docs.amd.com/en/latest/how-to/rocm-for-ai/index.html>

Profiling and Tooling

- ROCprofiler-SDK: <https://rocm.docs.amd.com/projects/rocprofiler-sdk/en/latest/>
- radeontop: <https://github.com/clbr/radeontop>
- ROCm SMI (amdsmi): <https://github.com/ROCm/amdsmi>

Closing Remarks

I appreciate the time you have already given to this during office hours. This document is intended to serve as a complete and transparent reference for the project scope, with all source repositories and documentation included so that any technical claim can be verified directly.

The project has grown in scope since our initial conversation. The addition of the GPU memory tiering manager (Layer 5), Flash Attention (Layer 8), and Triton support (Layer 9) reflects a deliberate effort to make the contribution genuinely useful to the AMD community at large *imes*-not just as a fix for one GPU, but as a complete open compute framework for a hardware class that has been neglected by the ecosystem.

I would welcome your feedback on the kernel architecture in particular, especially the memory tiering manager design and whether the DRM scheduler extension approach is sound. I plan to share initial benchmark data by mid-June.

All development will be publicly hosted on GitHub from day one.

Respectfully,

Parth Aditya

paditya2@wisc.edu

University of Wisconsin-Madison

B.S. Computer Science & Mathematics, December 2026